Assignment 1

Agents, Graph, Graph Searches and Implementation

Problem 1 Shopping Assistant Agents

A customer has a budget of $20. They may buy each item at most once from the catalog:

Item	Price	Satisfaction
Bread	5	6
Milk	4	7
Eggs	6	8
Chocolate	8	9

Description of the problem The problem is commonly known as the Knapsack problem. Since each item from the catalog can be chosen at most once, it can be categorized as a binary selection problem for each item, the decision take it or not. In fact, the formulation in Part B can be rewritten as a PLI (Programming Linear Integer) model.

The problem is NP-hard. However, in Part A, where its requested to formulate some rules for the agent is easy to apply an heuristic . This allows the problem to be solved efficiently, even though the solution obtained may not always be optimal. The heuristic is useful for defining the actions of the agent and provides a solution that is reasonably close to the best one.

For Part B, mathematical formulation is more appropriate, and the problem can be solved exactly using integer linear programming techniques.

Part A Reflex Agent:

- Define if-then rules
- Apply your rules step by step until the budget is reached or no more rules apply
- Show the basket, cost and satisfaction

Heuristic approach:

1. Evaluate the elements by their density satisfaction/cost
2. Order the elements in a decrescent way based on their density
3. Insert it in the bag until there is space

If then rules:

- The density of each item is evaluated in this way for the item i $d_i = {}^{s_i}/_{c_i}$ where s_i is the satisfaction of the item i and c_i the cost of the item i
- Each element I must be ordered in a way that $d_i > d_{i+1}$ if the inequality is not valid invert the order of the 2 elements
- The var p is the amount of budget that is going to be spend, n the number of items, x is a vector of item to be taken is 0 if the item i is not taken 1 instead :

for (i to n)
    If (budget-p)> c_i
        x_i=1
        p=p+c_i

Execution of the problem:

— Evaluation of the density $d_i = {}^{s_i}/_{c_i}$ —

$d_{bread} = 6/5 = 1.2$

$d_{milk} = 7/4 = 1.75$

$d_{eggs} = 8/6 = 1.\overline{3}$

$d_{chocolate} = 9/8 = 1.125$

— Ordering of the element —

Milk	Eggs	Bread	Chocolate
x_1	x_2	x_3	x_4

Milk	Eggs	Bread	Chocolate
x_1	x_2	x_3	x_4

— Execution of rules —

budget = 20 $

p=0	p=4	p=10	p=15
i=1	i=2	i=3	i=2
if true	if true	if true	if false
⇓	⇓	⇓	
x_1=1	x_2=1	x_3=1	20-15>8
p=4	p=10	p=15	

Basket = Milk, Eggs, Bread

Satisfaction = $\sum_{i=1}^{n} s_i x_i = 21$ Cost = $\sum_{i=1}^{n} c_i x_i = 15$

Part B Utility-Based Agent:

- Define a utility function
- Work out the basket that maximizes the satisfaction
- Compare the reflex agent's result in 2-3 sentences

The agent that have to respect a utility function could use his own feature for solve the problem and find the global best solution that in our case is better than the last one. This is because on the last solution we have followed an heuristic approach that is basic to teach to a machine respect to another that have to solve the PLI.

Matematical approach

PLI : $\max \sum_{i=1}^{4} x_i s_i$

$$\begin{cases} \sum_{i=1}^{4} x_i c_i \leq b \\ x_i \in \{0,1\} \end{cases}$$

best solution $x^* = [1\,0\,1\,1]$

- Basket = Milk, Bread, Chocolate
- Cost = $\sum_{i=1}^{n} c_i x_i = 17$
- Satisfaction = $\sum_{i=1}^{n} s_i x_i = 22$

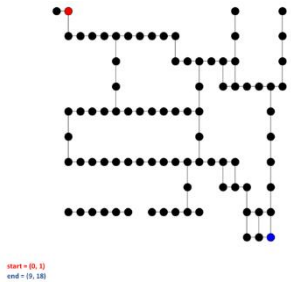
Problem 2 Implementation and compare search algorithms

- Find a graph rappresentation
- Implementation and compare of different search algoritms for the shortest path in a graph
- Analyze of the efficency, optipality and performance of each algorithm. Discussion strengths/weaknesses of each algorithm.

Graph Representation:

Common Graph Representations in Code:

- Adjacency Matrix
- Adjacency List
- Edge List



Adjency Matrix: This approach represents the graph as a binary matrix. In the case of an undirected graph, the matrix is symmetric. I found this approach to be a waste of memory for the maze problem, since it involves a large number of nodes but a relatively small number of edges. This results in a large matrix with very few entries set to 1, making it inefficient.

Adjency List: This method uses a dictionary (or similar structure) where each node is a key and its edges are stored as a list of values. Although handling this structure can be slightly more complex, it significantly reduces memory usage because only existing edges are stored. Depending on the problem, this approach can also improve search and modification costs.

Edge List : This representation maintains two separate lists: one for nodes and one for edges. It is the simplest to maintain and to add new nodes, but it increases the cost of visiting nodes and edges compared to the other methods.

Chosen Solution: I chose to implement the adjacency list because the graph structure is static, and I do not expect the graph to change during the program's execution. The adjacency list is stored in a JSON file. I attempted to use AI to assist in generating the graph, but this approach was unsuccessful, even when trying different prompts and few-shot learning techniques. The task proved too complex for AI, even with the help of coordinates.

Implementation of the algorithm :

```python
def dfs_path(node, target, visited=None, path=None):  2 usages
    if visited is None:
        visited = set()
    if path is None:
        path = []
    visited.add(node)
    path.append(node)
    if node == target:
        print(list(reversed(path)))
        return True  # stop further recursion
    for neighbor in graph_obj.graph[node]:
        if neighbor not in visited:
            if dfs_path(neighbor, target, visited, path):
                return True
    path.pop()  # backtrack
    return False
```

```python
def bfs_path(start, goal):  1 usage
    queue = deque([start])
    visited = set([start])
    parent = {start: None}  # mappa: each node points to its parent
    while queue:
        node = queue.popleft()
        if node == goal:
            # reconstruction of the path using parent dict
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            print (list(reversed(path)))
            return list(reversed(path))

        for neighbor in graph_obj.graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                parent[neighbor] = node
                queue.append(neighbor)
    return None
```

I also explored a solution where BFS was implemented using a queue containing pairs of [start, path]. However, for better readability and easier debugging, I decided to use the current implementation. Although the current approach requires an additional structure to store the path once the target node is reached, the alternative solution could have avoided this by maintaining the path directly within the queue.

Analysis and Comparison of Each Algorithm

DFS (Depth-First Search) considers a single path at a time and does not require structures like a queue. However, its main limitation is that it is strictly dependent on the way the graph is represented in the program. Specifically, the algorithm starts exploring from the first edge in the adjacency list and follows that path. Nodes are removed from the path when backtracking, and the search only stops when a leaf is reached and all adjacent nodes have been visited.

On the other hand, BFS (Breadth-First Search) always finds the shortest path in this case because the graph is unweighted. BFS explores nodes level by level, ensuring that the closest solution is found first.

One potential way to improve performance is to prioritize the target node, so that nodes that do not contribute to reaching the target are not evaluated. However, this approach can make the algorithm less reliable in finding a solution and harder to maintain.

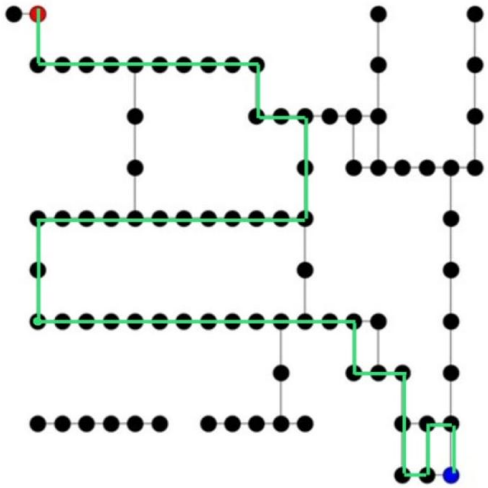
DFS is memory-efficient but path-dependent and may not find the shortest solution.

BFS guarantees the shortest solution but requires more memory.

Choice of algorithm depends on the graph's size, structure, and the importance of finding the shortest path versus memory efficiency.

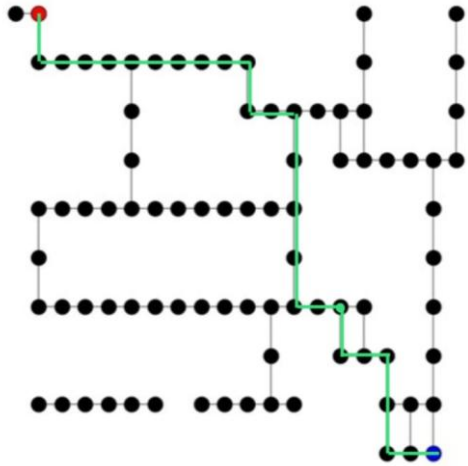
Out solution:

DFS path from (0,1) to (9,18):



['(0,1)', '(1,1)', '(1,2)', '(1,3)', '(1,4)', '(1,5)', '(1,6)', '(1,7)', '(1,8)', '(1,9)', '(1,10)', '(2,10)', '(2,11)', '(2,12)', '(3,12)', '(4,12)', '(4,11)', '(4,10)', '(4,9)', '(4,8)', '(4,7)', '(4,6)', '(4,5)', '(4,4)', '(4,3)', '(4,2)', '(4,1)', '(5,1)', '(6,1)', '(6,2)', '(6,3)', '(6,4)', '(6,5)', '(6,6)', '(6,7)', '(6,8)', '(6,9)', '(6,10)', '(6,11)', '(6,12)', '(6,13)', '(6,14)', '(7,14)', '(7,15)', '(7,16)', '(8,16)', '(9,16)', '(9,17)', '(8,17)', '(8,18)', '(9,18)']

BFS path from (0,1) to (9,18):



['(0,1)', '(1,1)', '(1,2)', '(1,3)', '(1,4)', '(1,5)', '(1,6)', '(1,7)', '(1,8)', '(1,9)', '(1,10)', '(2,10)', '(2,11)', '(2,12)', '(3,12)', '(4,12)', '(5,12)', '(6,12)', '(6,13)', '(6,14)', '(7,14)', '(7,15)', '(7,16)', '(8,16)', '(9,16)', '(9,17)', '(9,18)']